

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Объектно-ориентированное программирование»

«К защите допустить»
Руководитель курсового проекта
ассистент кафедры информатики
_____ А.Г. Бокун
____.____.2025

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
курсового проекта
на тему:
«РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ УЧЁТА ФИНАНСОВ»

БГУИР КП 6-05-0612-02 018 ПЗ

Выполнил студент группы 353503
РЯЗАНЦЕВ Алексей Владимирович

(подпись студента)

Курсовой проект представлен на
проверку _____.____.2025

(подпись студента)

Минск 2025

СОДЕРЖАНИЕ

Введение.....	5
1 Постановка задач для реализации поставленной задачи	6
1.1 Анализ существующих аналогов.....	6
1.2 Анализ предметной области	9
1.3 Общее описание приложения	9
1.4 Основные функциональные модули	10
2 Технологии программирования, используемые для решения поставленной задачи.....	11
2.1 Язык и платформа	11
2.2 Архитектура и библиотеки.....	16
2.3 Дополнительные технологии	18
3 Структура, архитектура и разработка приложения	19
3.1 Архитектура приложения.....	19
3.2 Разбиение компоненты и разработка приложения	21
3.3 Проектирование приложения.....	25
3.4 Тестирование приложения	32
Заключение	34
Список литературных источников	35

ВВЕДЕНИЕ

Данная работа посвящена разработке веб-приложения для учёта финансов, предоставляющего пользователям удобные инструменты для управления личными доходами и расходами. После анализа предметной области подтверждена актуальность и практическая значимость создания подобного программного обеспечения. Целевой аудиторией проекта являются частные пользователи, заинтересованные в контроле личных финансов, а также малые команды и индивидуальные предприниматели.

В рамках исследования были рассмотрены существующие аналоги, проведён сравнительный анализ их функциональных возможностей и интерфейсных решений. На основании полученных данных сформулированы ключевые требования к разрабатываемому приложению, направленные на достижение конкурентоспособности и высокого уровня удобства.

Целью курсового проекта является создание многофункционального веб-приложения, обеспечивающего возможность ведения учёта финансов с классификацией операций по категориям, поддержкой мультивалютности, регистрации и авторизации пользователей, а также удобного интерфейса для отображения финансовых показателей.

В ходе реализации проекта планируется рассмотреть архитектурные подходы к построению современных веб-систем, принципы REST-ориентированного API, работу с базами данных, механизмы аутентификации и авторизации, а также особенности взаимодействия фронтенд- и бэкенд-частей приложения. Предусматривается применение технологий Spring Boot, PostgreSQL, Redis, а также библиотеки React для клиентской части.

В результате выполнения работы будет разработан программный продукт, обладающий расширяемой архитектурой, наглядным пользовательским интерфейсом и возможностью последующего масштабирования. Приложение ориентировано как на индивидуальное использование, так и на внедрение в среду малого бизнеса.

Пояснительная записка оформлена в соответствии с СТП 01-2024 [1].

1 ПОСТАНОВКА ЗАДАЧ ДЛЯ РЕАЛИЗАЦИИ ПОСТАВЛЕННОЙ ЗАДАЧИ

1.1 Анализ существующих аналогов

MONEY, более 1 миллиона скачиваний. Это мобильное приложение существует с 2019 года и стабильно обновляется. MONEY выделяется своим минималистичным, интуитивным интерфейсом и ориентировано на быстрое добавление трат «в одно касание». Пользователю доступно ведение расходов по категориям, а также просмотр статистики с помощью диаграмм. При этом отсутствует возможность добавления доходов или мультивалютной поддержки, что делает приложение неполноценным для ведения общего бюджета. Основное внимание уделяется контролю трат, что делает его удобным для ежедневного использования, но не подходит для более комплексного учёта финансов.

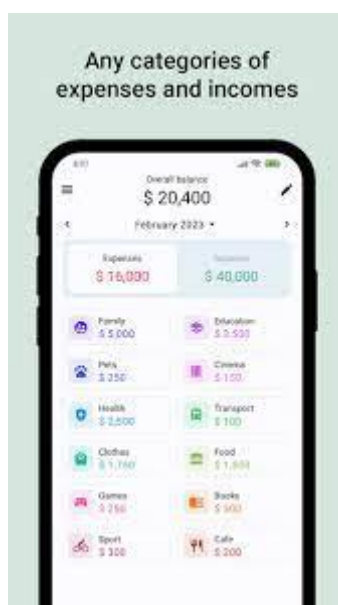


Рисунок 1.1 – MONEY

Wallet – личные финансы. Приложение выпускается чешской компанией BudgetBakers и с момента своего запуска в 2015 году стало одним из самых популярных инструментов для личного финансового учёта. Wallet предлагает широкие возможности: подключение банковских счетов, мультивалютность, планирование бюджета, финансовые цели, отчёты, импорт/экспорт данных, облачную синхронизацию и многое другое. Большинство функций доступны по подписке, но базовая версия также содержит обширный функционал. Приложение ориентировано как на продвинутых пользователей, так и на новичков. Из минусов: иногда перегруженный интерфейс, много рекламы в бесплатной версии и платный доступ к аналитике.

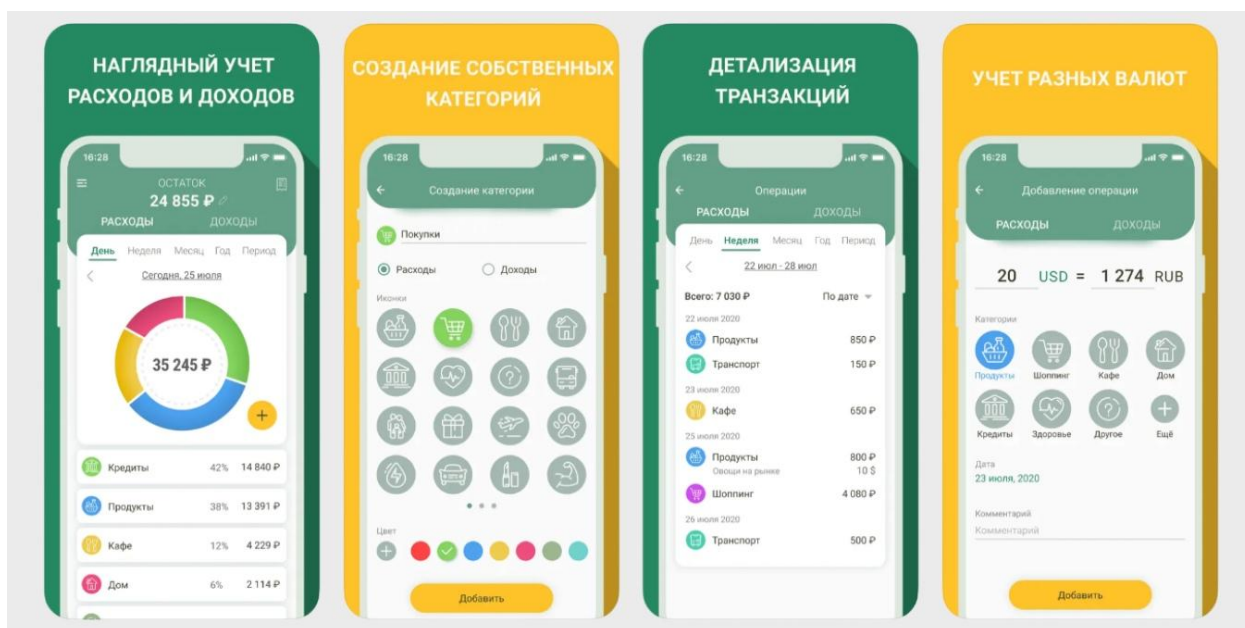


Рисунок 1.2 – Wallet – личные финансы

Spendee — это яркое, визуально привлекательное приложение, созданное с акцентом на дизайн и удобство визуального восприятия данных. Впервые представлено в 2014 году. Приложение позволяет вести бюджеты, подключать банковские счета, анализировать расходы и создавать совместные кошельки. Оно поддерживает мультивалютность, а также предлагает гибкую настройку категорий. Большая часть продвинутого функционала доступна в платной версии. Основные недостатки — наличие платной подписки, ограниченный функционал в бесплатной версии и иногда медленная работа при большом количестве данных.

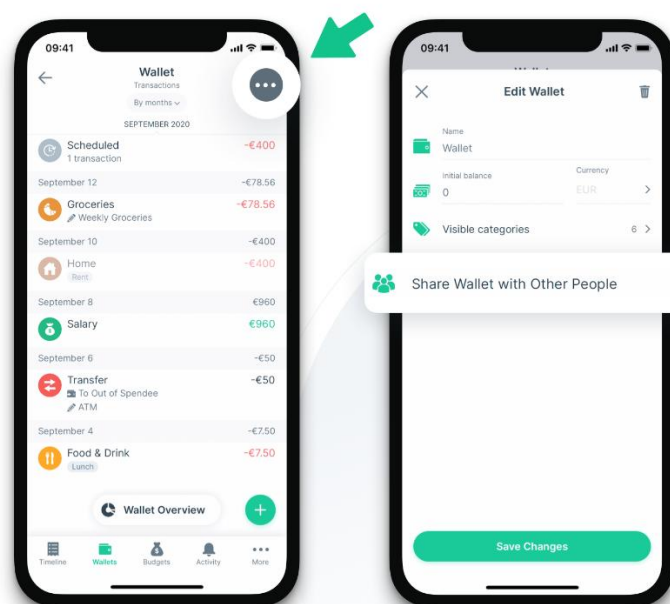


Рисунок 1.3 – Spendee

Money Manager считается одним из самых функциональных и «взрослых» приложений для учёта личных финансов. Оно активно развивается с 2012 года и предлагает такие возможности, как синхронизация с ПК, экспорт в Excel, графики по операциям, управление активами и пассивами, зашифрованное хранение данных. Приложение рассчитано на пользователей, привыкших вести подробный бюджет. Несмотря на слегка устаревший интерфейс, его высокая точность и гибкость сделали его фаворитом среди пользователей, предпочитающих контроль и детализацию. В

бесплатной версии доступны почти все функции, за исключением синхронизации с компьютером и отсутствия рекламы.

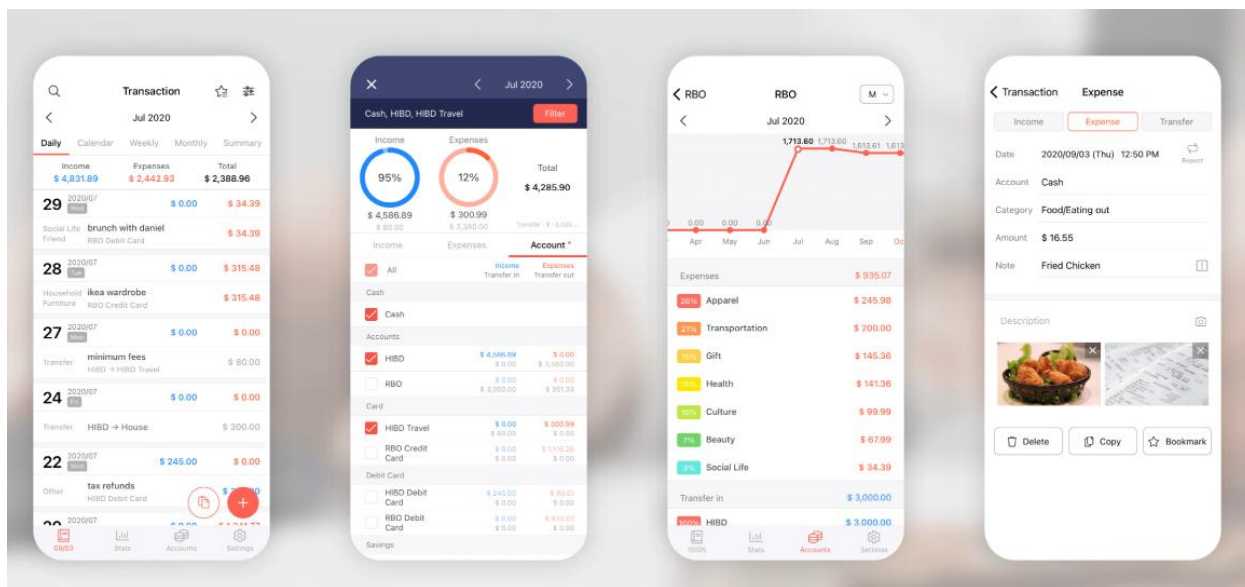


Рисунок 1.4 – Money Manager

1.2 Анализ предметной области

Проведён анализ предметной области веб-приложений для ведения личного финансового учёта на основе изучения существующих аналогов. Целью разрабатываемого программного продукта является объединение сильных сторон наиболее популярных решений, устранение их недостатков и реализация современного, расширяемого инструмента для управления доходами и расходами.

1.3 Общее описание приложения

Разрабатываемое приложение представляет собой веб-сервис с возможностью регистрации и авторизации пользователей, ведения персонального финансового учёта и анализа трат. Ключевыми функциями являются:

- Создание и редактирование финансовых аккаунтов с указанием;
- Ведение операций с классификацией по категориям;
- Работа с категориями (создание, изменение, удаление, разделение по типу: доход/расход);
- Аутентификация и авторизация пользователей с использованием JWT токена;
- Отображение общей финансовой информации в виде панелей внизу;
- Возможность сортировки и группировки информации (например, drag-

and-drop карточек аккаунтов и категорий);

- Современный адаптивный интерфейс для удобства использования на различных устройствах;

- Архитектура, обеспечивающая масштабируемость и возможность дальнейшего расширения функционала.

1.4 Основные функциональные модули

Управление аккаунтами включает в себя возможность добавления новых аккаунтов с указанием начального баланса и валюты. Пользователь может редактировать информацию об уже созданных аккаунтах, просматривать сводную информацию о них, а также удалять аккаунты при необходимости. Все аккаунты отображаются в виде карточек, обеспечивая наглядность и удобство восприятия. Также предусмотрена функция удаления связанных событий.

Управление операциями реализовано через создание доходных и расходных операций, которые прикрепляются к определённым аккаунтам и категориям. Пользователь может редактировать и удалять существующие операции, а также применять фильтрацию по различным параметрам, включая дату, категорию и сумму. Дополнительно выполняется автоматический расчёт общей суммы доходов и расходов за заданный период.

Управление категориями предоставляет пользователю возможность создавать новые категории с указанием их типа (доход или расход), редактировать и удалять существующие категории. Все категории визуализируются в виде плашек, разделённых по типу.

Пользовательская аутентификация и безопасность реализуются через систему регистрации и входа по электронной почте и паролю. Для защиты запросов используется механизм JWT-аутентификации. Все пользователи обладают базовой ролью, при этом архитектура приложения допускает расширение системы ролей и прав доступа в дальнейшем.

2 ТЕХНОЛОГИИ ПРОГРАММИРОВАНИЯ, ИСПОЛЬЗУЕМЫЕ ДЛЯ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

2.1 Язык и платформа

Java — это один из самых популярных и надёжных языков программирования в мире, используемый миллионами разработчиков для создания масштабируемых, надёжных и безопасных приложений. Благодаря своей зрелости и широкой экосистеме, Java идеально подходит для разработки серверной части веб-приложения, такого как система учёта финансов. Его строго типизированная модель и объектно-ориентированная природа позволяют создавать хорошо структурированный и поддерживаемый код, что особенно важно для сложных бизнес-приложений.



Рисунок 2.1 – Логотип языка программирования Java

Одним из ключевых преимуществ Java является богатый набор библиотек и фреймворков, а также широкая поддержка со стороны корпоративного сектора. Это делает Java отличным выбором для приложений, где требуются стабильность, безопасность, многопоточность и высокая

надёжность. В проекте финансового приложения это выражается, например, в корректной обработке транзакций, хранении пользовательских данных и надёжной работе с базой данных.

Кроме того, Java обладает кросс-платформенной природой благодаря своей виртуальной машине (JVM), что даёт возможность развёртывания приложения на любой серверной ОС — будь то Windows, Linux или macOS. Это особенно важно для гибкой настройки инфраструктуры и масштабирования проекта в будущем.

Также следует отметить, что Java активно развивается: появляются новые версии, которые расширяют синтаксис, улучшают производительность и предлагают современные подходы к разработке (такие как лямбда-выражения, стримы, pattern matching и т.д.). Всё это делает язык современным и удобным, несмотря на его возраст.



Рисунок 2.2 – Логотип фреймворка Spring Boot

Spring Boot — это один из самых мощных и широко используемых фреймворков в экосистеме Java, который предназначен для упрощённой и ускоренной разработки современных веб-приложений. Он построен на базе зрелого и гибкого фреймворка Spring Framework, который на протяжении более двух десятилетий является отраслевым стандартом в разработке корпоративных приложений. Благодаря Spring Boot разработчики могут сосредоточиться на реализации бизнес-логики, а не на конфигурации инфраструктурных компонентов, что особенно важно при ограниченных сроках разработки или в студенческих проектах.

Одним из ключевых преимуществ Spring Boot является автоматическая конфигурация: фреймворк самостоятельно подбирает необходимые зависимости и настройки в зависимости от содержимого проекта. Это

позволяет начать разработку буквально с нескольких строк кода, используя аннотацию `@SpringBootApplication`, и сразу же получить рабочее веб-приложение с встроенным сервером чаще всего Tomcat или Jetty. Таким образом, отпадает необходимость в ручной настройке XML-конфигураций или внедрении серверов приложений — всё работает из коробки.

Для финансового приложения, разрабатываемого в рамках данного проекта, Spring Boot предоставляет все необходимые инструменты:

- Spring Web для создания REST API и обработки HTTP-запросов;
- Spring Data JPA для работы с базами данных на основе ORM;
- Spring Security обеспечивает сильную аутентификацию, авторизацию и защиту данных;
- Spring Validation используется для точной валидации входных данных на уровне контроллеров и DTO;
- Spring Boot Actuator для мониторинга и управления приложением в продакшене.

Особенно следует отметить модуль Spring Security, который обеспечивает мощные средства для реализации механизма безопасности — как на уровне HTTP, сессии, токены, JWT, так и на уровне методов аннотации `@PreAuthorize`, `@Secured`. Для финансового приложения, где особое внимание уделяется защите пользовательских данных и финансовой информации, надёжная безопасность — приоритет номер один.

Spring Boot также отлично сочетается с современными практиками разработки:

- Поддерживает разделение на слои и чистую архитектуру, включая Onion Architecture;
- Легко интегрируется с Docker и системами непрерывной интеграции CI/CD;
- Имеет полноценную поддержку тестирования Spring Test, MockMvc, TestContainers;
- Предоставляет гибкую конфигурацию через файлы `application.yml` или `application.properties`.

Фреймворк активно поддерживается сообществом и компанией VMWare (ранее Pivotal), что обеспечивает регулярные обновления, исправления уязвимостей и постоянное внедрение лучших практик. Благодаря широкому распространению Spring Boot, существует множество готовых решений, tutorиалов и плагинов, что существенно упрощает обучение и разработку. Разработчику не нужно изобретать велосипед — достаточно правильно собрать компоненты из уже существующего, хорошо протестированного и документированного набора инструментов.

Важно также подчеркнуть, что Spring Boot — это не просто набор утилит, а целая экосистема, позволяющая реализовать как небольшие веб-сервисы, так и масштабируемые корпоративные платформы. Он используется в таких крупных компаниях, как Netflix, Amazon, Google, Alibaba и многих

других. Таким образом, выбор Spring Boot — это не только удобное, но и стратегически оправданное решение, соответствующее текущим промышленным стандартам разработки.

Именно поэтому Spring Boot был выбран в качестве основной серверной платформы для реализации веб-приложения по учёту финансов. Его богатый функционал, надёжность и активное сообщество делают его идеальным инструментом как для профессиональной разработки, так и для образовательных проектов, нацеленных на изучение современных подходов в индустрии программирования.



Рисунок 2.3 – Логотип СУБД PostgreSQL

PostgreSQL является одной из самых надёжных и функционально насыщенных систем управления базами данных, активно используемой как в академических проектах, так и в крупных коммерческих продуктах. Основанная на принципах открытого исходного кода и устойчивости к нагрузкам, PostgreSQL представляет собой не просто базу данных, а мощный инструмент хранения и обработки структурированной информации, полностью соответствующий требованиям современных приложений.

Одной из ключевых причин выбора PostgreSQL для данного проекта стало её строгое соответствие стандарту SQL и поддержка расширенной функциональности. В отличие от более легковесных решений PostgreSQL

обеспечивает полноценную поддержку транзакций, строгую типизацию, внешние ключи, каскадные обновления, триггеры и представления. Это критически важно в случае финансового приложения, где целостность данных, сохранение связей между сущностями и защита от потерь информации играют первоочередную роль.

Большое значение имеет и масштабируемость. PostgreSQL уверенно работает как на небольших учебных серверах, так и на кластерах с гигабайтами данных и высокой нагрузкой. Это означает, что приложение может развиваться, расширяться и усложняться без необходимости замены базовой инфраструктуры. Такое качество особенно важно для систем, предполагающих долгосрочную поддержку и возможность будущей интеграции с внешними сервисами и API.

Также PostgreSQL отличается гибкостью в работе с данными. Помимо реляционной модели PostgreSQL поддерживает хранение данных в формате JSON, работу с географическими данными через расширение PostGIS, а также позволяет использовать собственные функции и агрегаты. Это делает её универсальным инструментом, применимым как для классических CRUD-операций, так и для более сложной аналитики.

Отдельного упоминания заслуживает стабильность и надёжность системы. PostgreSQL активно развивается с начала 1990-х годов, за это время было накоплено огромное количество знаний, документации и успешных кейсов внедрения. Сообщество пользователей и разработчиков по всему миру вносит вклад в развитие и тестирование системы, что обеспечивает высокую надёжность и предсказуемое поведение даже в нестандартных сценариях использования.

Кроме того, PostgreSQL легко интегрируется со Spring Data JPA благодаря хорошо разработанному JDBC-драйверу и широкому сообществу. Настройка подключения и использование сущностей происходит интуитивно, а благодаря применению аннотаций и репозиториев вся логика взаимодействия с базой выносится на отдельный уровень, что упрощает сопровождение кода и разделение ответственности. Это особенно важно при использовании архитектуры, ориентированной на чистую структуру и модульность.

Таким образом, PostgreSQL представляет собой оптимальный выбор для проекта, требующего высокой стабильности, строгой целостности данных, надёжной поддержки транзакций и удобной интеграции с серверной частью, написанной на Java с использованием Spring Boot.

2.2 Архитектура и библиотеки

Для реализации веб-приложения по учёту финансов была выбрана многослойная архитектура на основе принципов разделения ответственности и слабой связности между компонентами. Центральное место в архитектуре занимает подход MVC — Model, View, Controller. Такой подход хорошо зарекомендовал себя в создании масштабируемых, поддерживаемых и легко расширяемых веб-приложений. Он позволяет чётко разделить бизнес-логику, представление и взаимодействие с пользователем, что особенно важно для системы, связанной с финансовыми данными и личной информацией пользователей.

Модель отвечает за работу с данными, включая сущности пользователей, аккаунтов, категорий и операций. Она взаимодействует с базой данных через репозитории и использует Data Transfer Objects для безопасной передачи информации между слоями. View в данном случае реализуется на фронтенде с помощью React, но в рамках серверной логики она представлена JSON-ответами REST API. Контроллеры принимают запросы от клиента, вызывают нужные сервисы и возвращают структурированные данные. Такой подход обеспечивает гибкость — можно менять внешний интерфейс без изменения серверной логики.

В качестве основного фреймворка используется Spring Boot. Это один из самых мощных и распространённых инструментов для разработки веб-приложений на Java, который предоставляет всё необходимое для создания production-ready приложений. Благодаря автоконфигурации и большому количеству интеграций Spring Boot позволяет сосредоточиться на бизнес-логике, а не на инфраструктуре. Например, вся настройка подключения к базе данных, кэширование и управление зависимостями может быть выполнено через простые конфигурационные файлы и аннотации.

Ключевым компонентом архитектуры выступает Spring Data JPA, который упрощает доступ к данным. Вместо написания вручную SQL-запросов можно использовать интерфейсы репозитория, описывая поведение через имена методов. Это не только ускоряет разработку, но и делает код более читаемым. При этом при необходимости всегда можно задать кастомные запросы на JPQL или нативном SQL.

Для обеспечения безопасности используется Spring Security. Он позволяет реализовать систему аутентификации и авторизации, а также защиту от типичных угроз, таких как CSRF, XSS и атаки методом перебора. В рамках проекта реализована регистрация и вход в систему с использованием JWT-токенов. Это обеспечивает stateless-аутентификацию, что особенно важно в условиях REST-архитектуры. Токен хранится на клиенте и прикрепляется к каждому запросу, позволяя серверу без дополнительного состояния определять, кто именно обращается к ресурсу.

Отдельного внимания заслуживает использование DTO — объектов

передачи данных. Это отдельный слой между моделью и контроллером, обеспечивающий безопасность, снижение связности и удобство сериализации. DTO используются для всех входящих и исходящих запросов, что позволяет изолировать внутреннюю структуру доменных моделей от внешнего API. Такой подход особенно важен в системе, где есть чёткое разграничение доступа между пользователями и необходима валидация данных на уровне контроллера.

Также в проекте активно используется Bean Validation через аннотации, такие как `NotNull`, `Size`, `Email`, которые позволяют проверять входящие данные до их обработки. Это минимизирует количество ошибок и улучшает пользовательский опыт.

Проект реализован с учётом принципов SOLID и архитектуры Onion. Это позволяет не только разделить ответственность, но и легко добавлять новые модули — например, поддержку конвертации валют, синхронизацию с банками или экспорт в формат Excel. Использование интерфейсов и инъекции зависимостей через Spring Container делает систему гибкой и легко тестируемой.

Благодаря использованию модульного подхода, соблюдению чистой архитектуры и современных технологий экосистемы Java, приложение не только соответствует требованиям надёжности и безопасности, но и легко масштабируется для добавления новых функций в будущем. Это особенно важно при работе с финансами, где высоки требования к точности, доступности и защите данных.

Одной из вспомогательных, но крайне полезных библиотек, применяемых в проекте, является Lombok. Её основное предназначение — уменьшение шаблонного кода и повышение читаемости моделей и DTO. В типичном Java-проекте значительная часть кода уходит на генерацию геттеров, сеттеров, конструкторов, методов `equals`, `hashCode` и `toString`. Lombok позволяет избавиться от этого, используя простые аннотации, такие как `Data`, `Getter`, `Setter`, `NoArgsConstructor`, `AllArgsConstructor` и `Builder`.

В рамках нашего приложения это особенно удобно при работе с сущностями, такими как `User`, `Account` или `Category`. Вместо того чтобы вручную прописывать все поля и методы, достаточно указать нужные аннотации, и Lombok сгенерирует всё необходимое во время компиляции. Это упрощает сопровождение кода и делает его компактнее и нагляднее.

Также библиотека отлично сочетается с остальными инструментами Spring. Например, аннотация `RequiredArgsConstructor` позволяет автоматически создать конструктор для `final`-полей, что удобно при внедрении зависимостей через конструктор. При использовании DTO с аннотацией `Builder` можно легко и безопасно создавать объекты с необходимыми полями без длинных конструкторов и лишнего кода.

Таким образом, Lombok помогает сконцентрироваться на логике приложения, не отвлекаясь на рутинное написание однотипных методов. Это

особенно важно в крупном проекте, где важна чистота кода, единообразие и минимизация технического долга.

2.3 Дополнительные технологии

Для повышения надёжности и производительности веб-приложения были использованы дополнительные технологии, направленные на обеспечение кэширования, безопасности, взаимодействия между клиентской и серверной частями системы, а также удобства развёртывания.

Redis применён в качестве механизма кэширования. Его использование позволило сократить количество обращений к постоянному хранилищу данных при повторяющихся запросах. Это особенно актуально для информации, не изменяющейся в течение короткого времени, например, списка категорий расходов или сведений о счетах. Кэширование с использованием Redis повышает скорость отклика интерфейса и снижает нагрузку на серверную часть приложения.

JWT (JSON Web Token) используется для аутентификации пользователей. Каждый авторизованный пользователь получает уникальный токен, передаваемый клиентом при последующих запросах. Такой подход исключает необходимость хранения пользовательских сессий на сервере и позволяет надёжно защищать маршруты и ресурсы с ограниченным доступом, обеспечивая простой механизм проверки подлинности.

CORS (Cross-Origin Resource Sharing) настроен для управления безопасностью при взаимодействии клиентской и серверной части. Поскольку приложение построено с разделением интерфейсной и серверной логики, требуется явное разрешение на выполнение кросс-доменных запросов. Это обеспечивается настройкой допустимых источников, методов и заголовков.

REST API реализует обмен данными между компонентами системы. Использование HTTP-запросов и единообразной структуры URL упрощает интеграцию новых модулей, поддерживает масштабируемость и стандартизирует взаимодействие с такими сущностями, как категории, счета и операции. Поведение API предсказуемо и соответствует принципам REST-подхода.

Docker используется для контейнеризации серверной и вспомогательной инфраструктуры. Благодаря этому развёртывание приложения осуществляется быстро и одинаково на любой среде. Использование контейнеров позволяет изолировать компоненты, упрощает масштабирование, обновление и миграции. Контейнеры включают все необходимые зависимости, что устраняет проблему различий между средами разработки и эксплуатации.

3 СТРУКТУРА, АРХИТЕКТУРА И РАЗРАБОТКА ПРИЛОЖЕНИЯ

3.1 Архитектура приложения

В разрабатываемом веб-приложении применена многослойная архитектура, обеспечивающая разделение ответственности и повышение сопровождаемости проекта. Такое архитектурное решение позволяет изолировать пользовательский интерфейс, бизнес-логику и работу с данными, что облегчает тестирование, повторное использование компонентов и масштабирование системы.

Архитектура включает четыре ключевых уровня: пользовательский интерфейс, слой контроллеров, сервисный слой и слой доступа к данным. Каждый из уровней выполняет строго определенные функции и взаимодействует с соседними через заранее определенные интерфейсы, что соответствует принципам слабой связанности и высокой связности внутри слоёв.

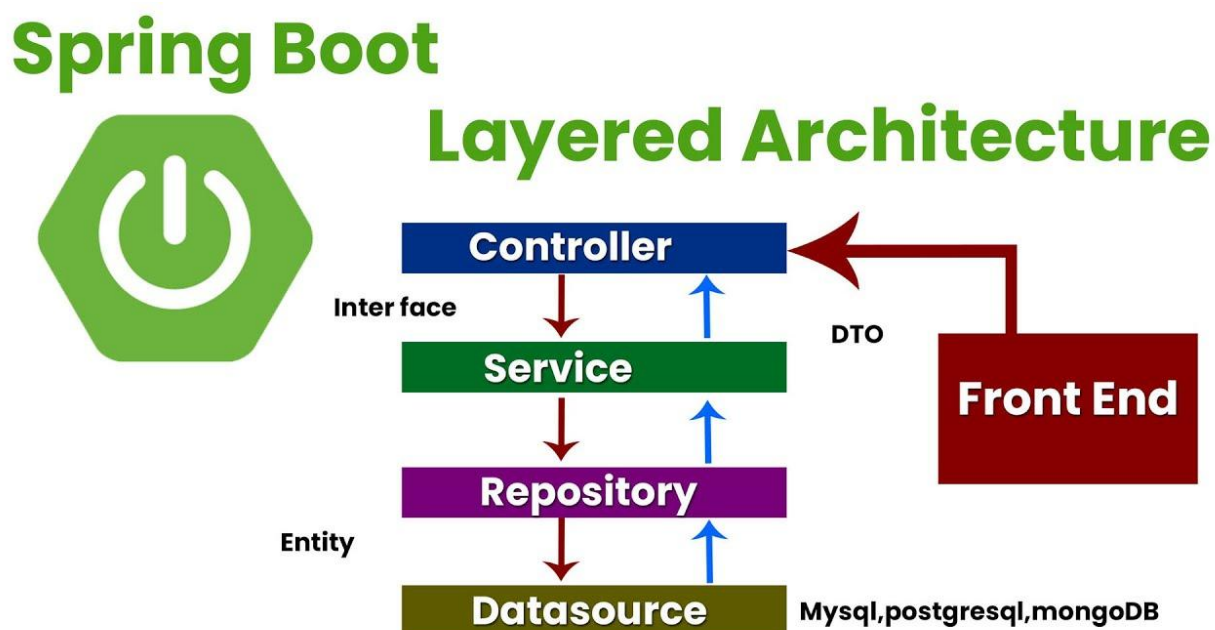


Рисунок 3.1 – Схема типичной архитектуры web приложения

Основными слоями приложения являются: слой пользовательского интерфейса, слой контроллеров, сервисный слой и слой доступа к данным. Каждый из этих компонентов выполняет строго ограниченную роль и взаимодействует с другими через определённые контракты.

Пользовательский интерфейс реализован с использованием React — популярной JavaScript-библиотеки для создания динамических одностраничных

приложений. Он выполняет функцию отображения информации, полученной от сервера, и обеспечивает двустороннее взаимодействие с пользователем. Пользователь совершает действия, такие как ввод данных, нажатие кнопок или переключение между вкладками, и интерфейс формирует соответствующие HTTP-запросы к серверу. Отдельное внимание при разработке было уделено управлению состоянием и компонентной структуре, что позволило обеспечить масштабируемость и расширяемость фронтенд-части. Используются библиотека `axios` для взаимодействия с backend и `Material UI` для построения адаптивного интерфейса.

Слой контроллеров построен на основе `Spring MVC`. Контроллеры служат точками входа во внутреннюю часть приложения. Они принимают входящие запросы от клиента, интерпретируют параметры, проводят базовую валидацию и перенаправляют данные в соответствующие сервисы. Результаты выполнения передаются обратно клиенту в виде JSON-объектов. Контроллеры также отвечают за отображение кодов ошибок и сообщения об исключениях, в случае некорректной работы сервисов или предоставления некорректных данных.

Центральным звеном архитектуры является сервисный слой. Именно здесь сосредоточена основная бизнес-логика приложения. Сервисы реализуют обработку запросов, проверку правил, преобразование и агрегацию данных, управление транзакциями, а также координацию взаимодействия между различными частями системы. Такой подход позволяет изолировать логику от внешних источников, обеспечивая независимость бизнес-функциональности от деталей реализации хранения или представления данных. В случае необходимости расширения функциональности сервисный слой может быть дополнен новыми методами без затрагивания остальных компонентов.

Доступ к данным осуществляется через репозитории, реализованные с использованием `Spring Data JPA`. Репозитории представляют собой абстракции, инкапсулирующие логику взаимодействия с базой данных. Каждый репозиторий отвечает за одну сущность или группу связанных сущностей и предоставляет интерфейсы для стандартных операций — создания, чтения, обновления и удаления (CRUD). Такой подход избавляет от необходимости вручную писать SQL-запросы и позволяет сосредоточиться на логике приложения. В качестве основной базы данных используется `PostgreSQL`, что обеспечивает надежность, совместимость с транзакциями и хорошую масштабируемость. Вспомогательно используется `Redis` для кэширования данных, ускоряя доступ к наиболее часто используемым объектам и снижая нагрузку на основную базу данных.

Дополнительно в архитектуре используется подход разделения данных с помощью DTO (`Data Transfer Objects`). Это позволяет отделить внутреннюю структуру доменных моделей от того, что передается на клиент или между слоями приложения. Таким образом, исключается утечка деталей реализации и обеспечивается безопасность при передаче информации.

Такое слоистое построение приложения соответствует принципам SOLID, способствует единообразию кода и упрощает процесс тестирования. Каждый слой может быть протестирован независимо, что особенно важно в случае масштабного проекта. Кроме того, подобная архитектура позволяет легко интегрировать внешние API или модули, такие как платёжные шлюзы, внешние сервисы аутентификации или системы аналитики.

В целом, выбранная архитектура обеспечивает высокую гибкость и стабильность веб-приложения. Она позволяет вести параллельную разработку клиентской и серверной частей, адаптировать проект под новые требования и упрощает поддержку в долгосрочной перспективе.

3.2 Разбиение на компоненты и разработка приложения

Так как создаваемое веб-приложение обладает значительным объёмом бизнес-логики, архитектура была построена с учётом разделения ответственности между слоями. В проекте используется подход, основанный на Onion-архитектуре, предполагающей выделение трёх ключевых слоёв: слоя представления (UI), слоя приложения (application/service layer) и доменного слоя (domain). Также выделяется отдельный слой инфраструктуры (data/infrastructure), реализующий детали взаимодействия с внешними ресурсами, такими как база данных или сторонние сервисы. Такое разделение позволяет избежать дублирования кода, инкапсулировать сложную бизнес-логику и значительно упростить тестирование приложения.

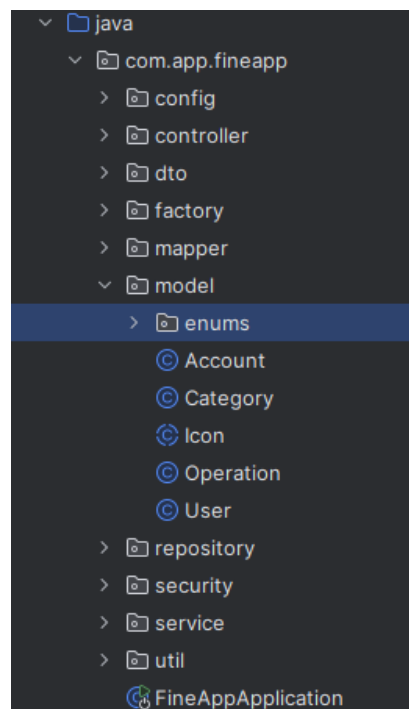


Рисунок 3.2 – Структура доменного слоя

Центральное место занимает доменный слой. Он содержит основные бизнес-сущности, агрегаты, интерфейсы репозитория и контрактные интерфейсы для сервисов, определяющие бизнес-логику приложения. Доменные модели отражают структуру бизнес-объектов, таких как пользователь, аккаунт, категория или транзакция. Принцип инверсии зависимостей соблюдается за счёт того, что доменный слой не зависит от технических деталей, реализованных в нижележащих слоях, включая работу с базой данных или формат обмена данными по сети. Напротив, все внешние зависимости определяются в виде интерфейсов, что обеспечивает слабую связанность и высокую модульность архитектуры.

Интерфейсы репозитория и сервисов определяются в доменном слое, а их конкретные реализации размещаются в слое инфраструктуры. Такой подход позволяет организовать независимость бизнес-логики от механизмов хранения и извлечения данных. Модели, используемые в доменном слое, отделены от моделей, применяемых в инфраструктурном слое (например, сущности JPA), что обеспечивает полную изоляцию бизнес-объектов от технических аспектов. Это позволяет избежать каскадного изменения бизнес-кода при модификации структуры базы данных или при переходе на другие технологии хранения.

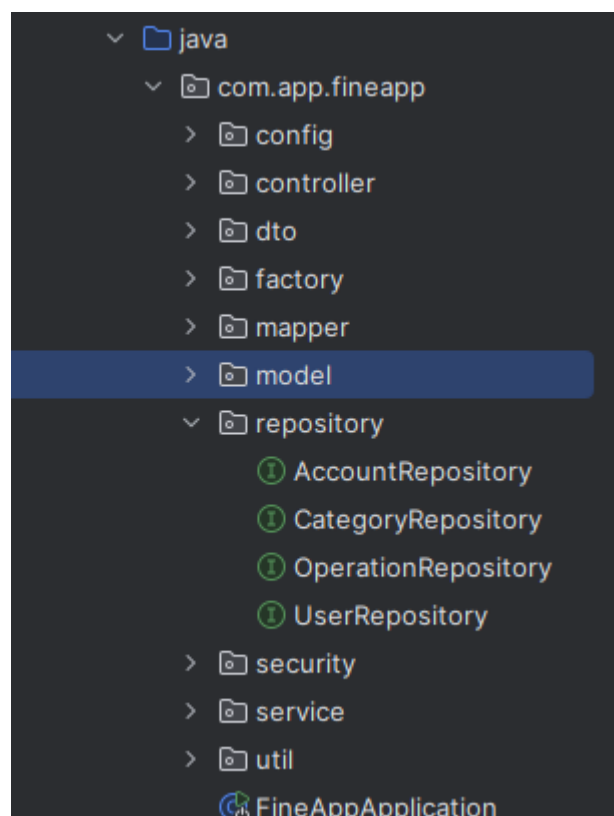


Рисунок 3.3 – Структура слоя данных

Реализация взаимодействия с базой данных производится в инфраструктурном слое. Здесь определены конкретные реализации репозитория с использованием Spring Data JPA. Этот инструмент позволяет автоматически генерировать реализацию на основании сигнатур интерфейсов, значительно сокращая количество шаблонного кода. В данном слое также возможно подключение дополнительных компонентов, таких как кэш Redis или интеграция с внешними API. Таким образом, можно чётко разделить локальную и внешнюю бизнес-логику. Помимо базовых хранилищ, слой инфраструктуры включает также преобразователи моделей (мапперы), необходимые для преобразования между доменными объектами и объектами передачи данных (DTO).

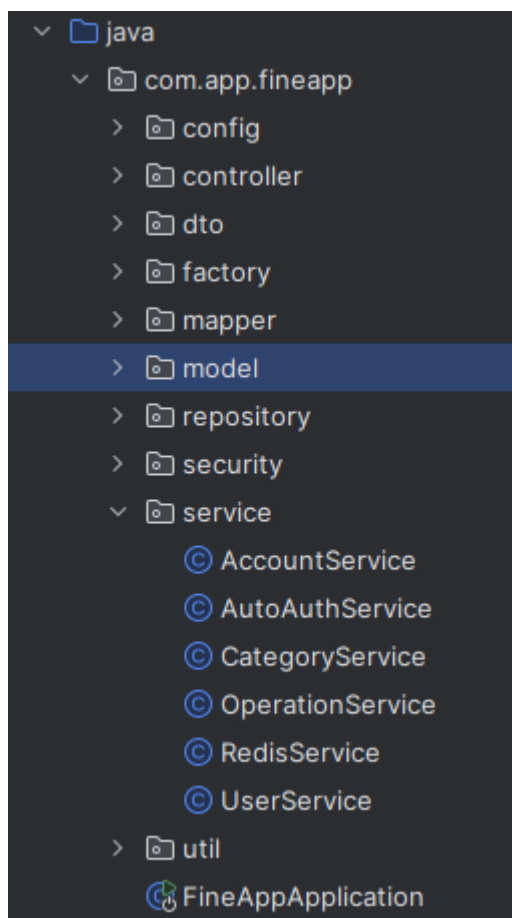


Рисунок 3.4 – Структура слоя бизнес логики

Слой представления в рамках веб-приложения разделяется на клиентскую и серверную составляющие. На стороне клиента используется фреймворк React, обеспечивающий построение компонентов пользовательского интерфейса и управление состоянием. React-компоненты отвечают за отображение данных, полученных с сервера, и взаимодействие с пользователем: ввод данных, навигация, вызов API. Архитектурно интерфейсный слой использует паттерн умных и глупых компонентов —

логика взаимодействия с сервером и управление состоянием сосредоточены в контейнерных компонентах, тогда как отображение данных поручено компонентам-обёрткам.

На стороне сервера в представление входит слой контроллеров (Spring MVC). Контроллеры принимают HTTP-запросы от клиента, выполняют валидацию параметров и делегируют обработку сервисному слою. Результаты возвращаются в виде JSON-объектов, что позволяет эффективно взаимодействовать с фронтендом. Контроллеры, как правило, не содержат бизнес-логики — они служат посредниками между клиентом и логикой приложения.

Во всей архитектуре применён паттерн MVVM в адаптированной форме. Состояния хранятся в клиентском состоянии React-компонентов, события пользователя инициируют вызовы к API, а логика обработки запросов выполняется в сервисах на сервере. Для каждого компонента можно выделить собственное состояние, обработчики событий и реакцию на изменения данных. Это позволяет достичь предсказуемости поведения интерфейса и облегчить отладку. События, возникающие на клиенте, отправляются как HTTP-запросы, обрабатываются контроллерами и соответствующими методами сервисного слоя. Состояния React компонентов обновляются в соответствии с полученными от сервера ответами.

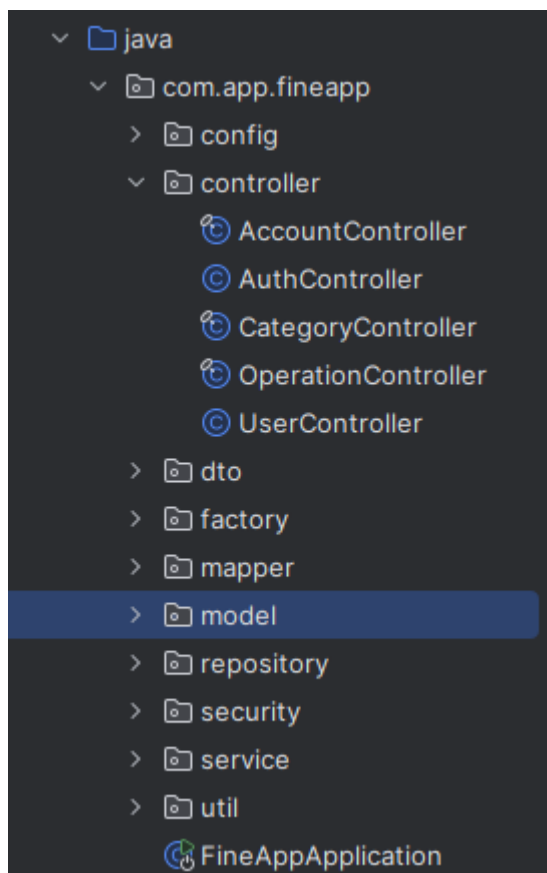


Рисунок 3.8 – Структура слоя представления

3.3 Проектирование приложения

Первым элементом, с которым взаимодействует пользователь при заходе на веб-приложение, является стартовая страница. На ней отображается приветственное окно с предложением авторизоваться или пройти регистрацию, если пользователь впервые посещает сервис. Такое разделение потоков входа в систему позволяет обеспечить как быструю повторную аутентификацию, так и полноценное онбординг-взаимодействие с новым пользователем.

Главная цель стартовой страницы — простота, понятность и акцент на действие. Именно поэтому интерфейс содержит только самые необходимые элементы: логотип или название проекта, краткий слоган и две основные кнопки — Войти и Зарегистрироваться. При этом для зарегистрированных пользователей доступна функция автологина, если соответствующий пользователь уже сохранён в локальном хранилище.

Такой подход позволяет упростить взаимодействие с системой, минимизировать количество шагов до начала работы и положительно влияет на вовлечённость новых пользователей. Авторизация осуществляется по email и паролю, при этом предусмотрена валидация ввода и отображение соответствующих ошибок в случае неправильного ввода или отсутствия пользователя. Регистрация аналогично проверяет наличие уже существующего аккаунта по email.

Добро пожаловать в FineApp!

Управляйте своими финансами легко и удобно



Рисунок 3.9 – Стартовая страница с окнами авторизации и регистрации

После того как пользователь нажимает на кнопку входа или регистрации на стартовой странице, он попадает на соответствующую страницу с формой. Каждое из окон оформлено в едином стиле, чтобы сохранить визуальное и логическое единство интерфейса приложения. Переход между страницами реализован таким образом, чтобы пользователь мог свободно переключаться между формами регистрации и входа, а также при необходимости вернуться обратно на стартовую страницу, где описаны ключевые преимущества сервиса.

На странице авторизации пользователь вводит свой email и пароль, после чего происходит проверка данных. Интерфейс реализован таким образом, чтобы максимально сократить количество отвлекающих элементов и сосредоточить внимание пользователя на выполнении целевого действия — входа в систему. Если пользователь вводит некорректные данные, на экране появляется соответствующее сообщение с описанием ошибки (например, Неверный email или пароль). В случае успешного ввода пользователь перенаправляется на основную страницу приложения, где отображается его персональная информация и связанные с ним данные (например, список аккаунтов и категорий).

Для упрощения навигации предусмотрена текстовая ссылка, позволяющая перейти на страницу регистрации, если пользователь ещё не зарегистрирован. Также реализован переход на стартовую страницу, позволяющий вернуться к первоначальному виду сайта.

Вход

Email

Пароль

ВОЙТИ

[Нет аккаунта? Зарегистрироваться](#)

Рисунок 3.10 – Страница авторизации пользователя

На странице регистрации пользователь указывает адрес электронной почты и пароль. При этом реализована предварительная проверка существования пользователя по email: если такой email уже зарегистрирован, пользователь получает соответствующее уведомление. Если email уникален, происходит создание нового пользователя в системе, и он автоматически авторизуется, получая доступ к основной части приложения.

Форма регистрации дополнена сообщением о политике безопасности хранения пароля. Также, как и на странице входа, доступны переходы на стартовую страницу и на страницу входа, если пользователь понял, что уже имеет аккаунт.

Для корректного UX важную роль играет однозначная обратная связь:

пользователь в любой момент может понять, на какой странице он находится и какие действия доступны. Навигация интуитивно понятна, а оформление страниц помогает пользователю быстро ориентироваться в системе.

Регистрация

Email

Пароль

Повтор пароля

ЗАРЕГИСТРИРОВАТЬСЯ

[Уже есть аккаунт? Войти](#)
[← На главную](#)

Рисунок 3.11 – Страница регистрации нового пользователя

После успешной авторизации пользователь попадает на основную страницу приложения, которая представляет собой центр взаимодействия с функциональными разделами. Интерфейс данной страницы построен по принципу вкладок и содержит четыре основные секции, доступные через нижнюю панель навигации. Такой подход обеспечивает быстрый доступ ко всем ключевым возможностям системы и делает навигацию интуитивно понятной даже для новых пользователей.

В нижней части экрана располагается панель с четырьмя вкладками, каждая из которых соответствует отдельной функциональной области приложения:

- Аккаунты – отображает список финансовых счетов пользователя с текущими остатками. Пользователь может создавать новые аккаунты, редактировать и удалять существующие. Также реализована возможность сортировки аккаунтов и просмотра детальной информации.

- Категории – включает категории доходов и расходов, разделённые на две колонки. Поддерживается drag-and-drop внутри колонок, что упрощает управление структурой бюджета. Категории снабжены цветами, названиями и отображают суммы за текущий месяц.

– Операции – отображает список всех финансовых операций. В этом разделе пользователь может фильтровать операции по дате, категории или аккаунту, добавлять новые записи, а также редактировать или удалять существующие.

Переход между вкладками осуществляется мгновенно, без перезагрузки страницы, благодаря использованию асинхронной загрузки данных. Это обеспечивает высокую отзывчивость интерфейса и комфорт при использовании приложения.

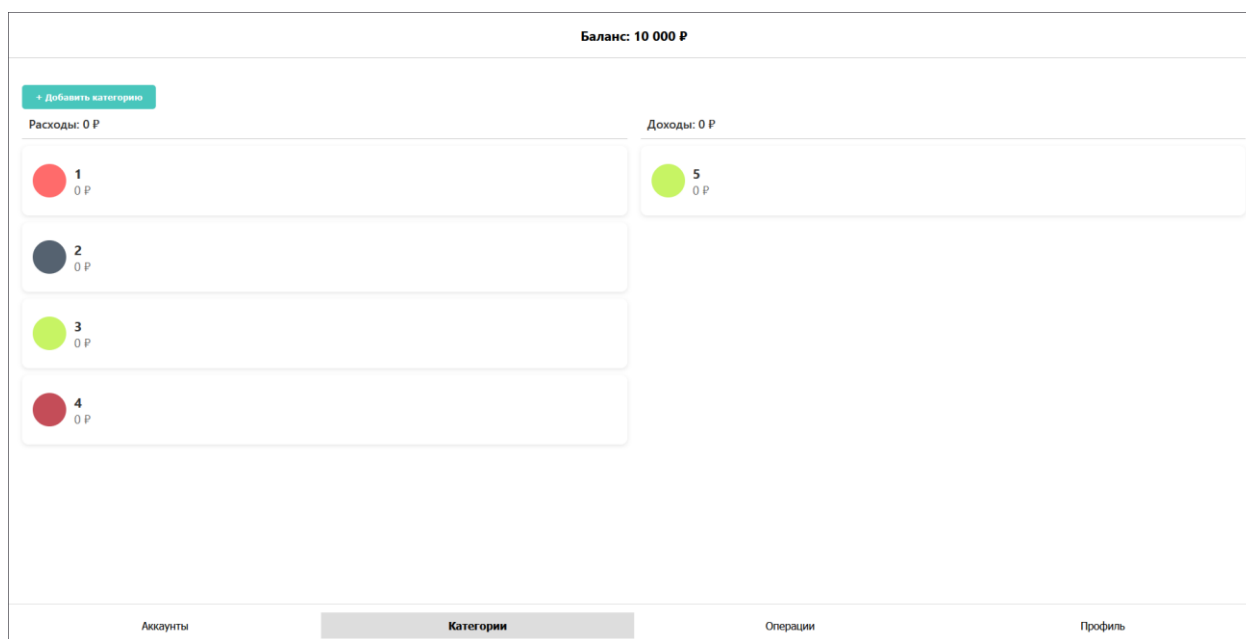


Рисунок 3.12 – Главная страница приложения с нижней панелью вкладок

Помимо функциональных вкладок, на главной странице также имеется кнопка вызова пользовательского меню. Меню открывается в виде всплывающей панели, в которой отображается основная информация о текущем пользователе — адрес электронной почты, имя (если указано) и статус. Также в меню представлены две важные опции:

– Выход из системы. Завершает текущую сессию и перенаправляет пользователя на стартовую страницу приложения. При этом JWT-токен сбрасывается на стороне клиента, а данные о сохраненном пользователе удаляются из файла.

– Удаление аккаунта. При подтверждении полностью удаляет все данные пользователя из системы, включая связанные аккаунты, категории и операции. Данная операция является необратимой и сопровождается соответствующим предупреждением.

Меню пользователя можно вызвать в любой момент, независимо от того, в каком разделе приложения находится пользователь, что делает доступ к ключевым действиям всегда быстрым и удобным.

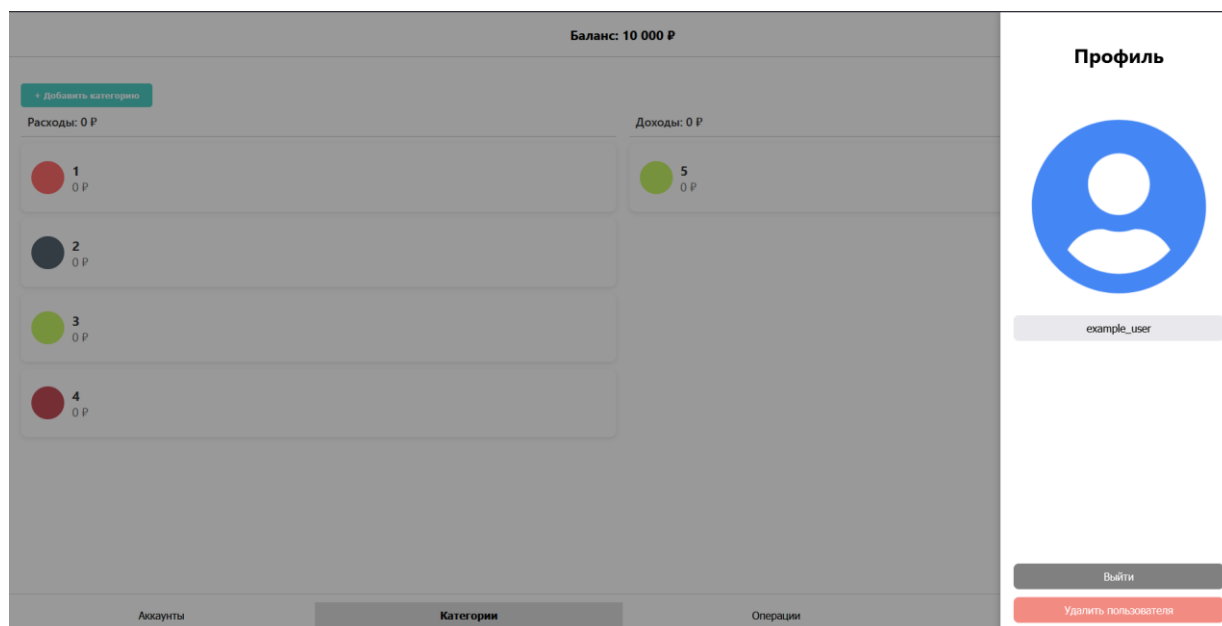


Рисунок 3.13 – Всплывающее меню пользователя

Для повышения удобства использования веб-приложения и устранения необходимости повторного ввода данных при каждом посещении реализован механизм автоматической авторизации. Такой подход позволяет пользователю оставаться в системе до момента явного выхода или удаления аккаунта, обеспечивая плавный пользовательский опыт.

Механизм авто-авторизации основан на сохранении информации о пользователе в локальном хранилище браузера. На практике используется отдельный JSON-файл, создаваемый на стороне клиента после успешной авторизации или регистрации. В этом файле сохраняются ключевые данные пользователя, включая токен и его идентификатор. Данные могут быть размещены, например, в `localStorage`, `sessionStorage` или во внутреннем состоянии клиентского приложения, в зависимости от выбранной реализации. Основные этапы работы механизма следующие:

- Сохранение состояния пользователя. После успешной авторизации (через логин) или регистрации пользователя, данные о нём сериализуются в формате JSON и записываются в локальное хранилище браузера. Это позволяет при последующих загрузках страницы восстановить сессию без необходимости ручного ввода данных;

- Проверка наличия авторизованного пользователя. При каждом посещении страницы входа срабатывает скрипт, автоматически выполняющий проверку локального файла с сохранёнными данными. Если файл существует и содержит валидную информацию о пользователе (включая действительный токен), система автоматически восстанавливает сессию, минуя ручной ввод данных. Таким образом, пользователь сразу перенаправляется в основное окно приложения, минуя форму входа;

– Очистка состояния при выходе. При выходе пользователя из системы либо при удалении аккаунта вызывается процедура очистки локального файла. Это обеспечивает безопасность и предотвращает несанкционированный доступ к данным после завершения пользовательской сессии.

Такой подход реализует локальное запоминание активного пользователя и обеспечивает высокий уровень комфорта за счёт мгновенного входа без ввода логина и пароля при повторных визитах.

Одним из ключевых элементов пользовательского интерфейса веб-приложения является отдельная страница управления категориями. Категории в системе предназначены для логического разделения операций пользователя по смысловым группам и упрощают навигацию и анализ расходов и доходов.

Страница категорий организована в виде двух визуально разделённых колонок: одна содержит категории, относящиеся к доходам, другая – к расходам. Такой подход позволяет пользователю мгновенно ориентироваться в назначении той или иной категории и упрощает управление финансовыми потоками.

Каждая категория представлена в виде интерактивной плашки, визуальный стиль которой может быть настроен пользователем. В частности, доступна кастомизация цвета карточки, что позволяет выделять важные категории или визуально структурировать информацию по личным предпочтениям. Цвет задаётся при создании категории и может быть изменён в любой момент.

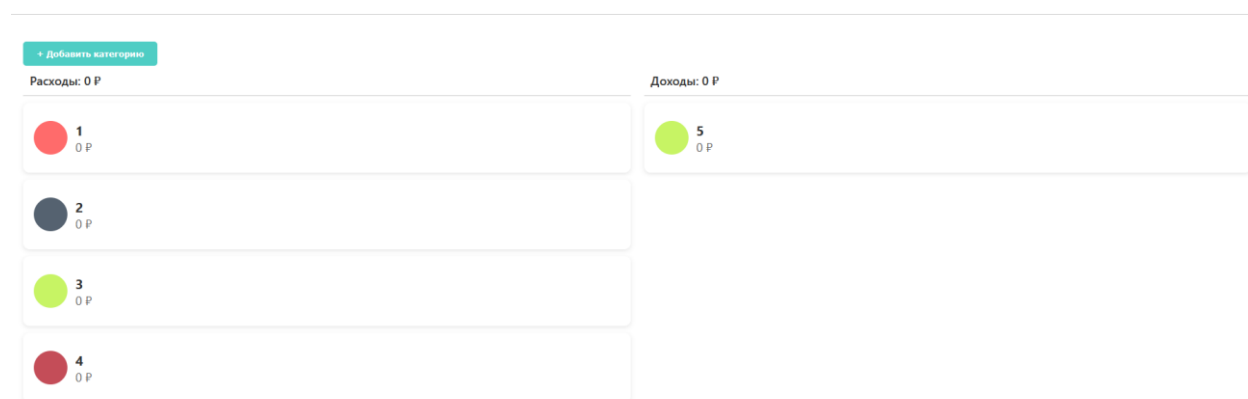


Рисунок 3.14 – Интерфейс страницы категорий с разделением на доходы и расходы

Ключевая функциональность страницы категорий заключается в возможности создания операции непосредственно через взаимодействие с плашками. При нажатии на конкретную категорию открывается специальное всплывающее меню, через которое пользователь может создать новую

операцию. В этом меню предлагается:

- выбрать счёт, с которым будет связана операция;
- ввести сумму перевода.

Далее логика зависит от типа выбранной категории:

- если выбрана категория доходов, то указанная сумма зачисляется на выбранный счёт;
- если выбрана категория расходов, то указанная сумма списывается с выбранного счёта.

Созданная операция сохраняется в базе данных и становится доступной для дальнейшего редактирования или удаления через интерфейс управления операциями. Таким образом, каждое взаимодействие с категорией формирует осмысленную транзакцию, которая немедленно отражается на балансе соответствующего счёта.

Такой подход к взаимодействию с категориями повышает скорость и удобство создания операций, снижает количество промежуточных действий со стороны пользователя и делает интерфейс более интуитивно понятным.

Рисунок 3.15 – Всплывающее меню создания операции через категорию

3.4 Тестирование приложения

Тестирование является неотъемлемой частью процесса разработки веб-приложения, обеспечивая его стабильность, надёжность и соответствие функциональным требованиям. В рамках данного проекта применяется комбинация современных инструментов и подходов, позволяющих эффективно проверять работу ключевых компонентов серверной части приложения, реализованной на базе Spring Boot.

Для модульного и интеграционного тестирования используется фреймворк JUnit 5. Каждый тестовый метод помечается аннотацией `@Test`, что позволяет точно определить, какой участок логики подлежит проверке. Кроме того, в проекте активно применяется библиотека Mockito, обеспечивающая возможность создания поддельных (mock) зависимостей, таких как репозитории и сторонние сервисы. Это позволяет изолировать тестируемый компонент от внешних ресурсов и обеспечить быструю и надёжную проверку бизнес-логики.

Поведение mock-объектов настраивается с помощью методов `when(...)` и `thenReturn(...)`, а проверка вызовов — через `verify(...)`. Такой подход позволяет удостовериться, что взаимодействие между слоями приложения происходит корректно и предсказуемо. Особенно важно это при тестировании сервисного слоя, в котором сосредоточена основная логика обработки данных.

Для тестирования контроллеров и REST API применяется аннотация `@WebMvcTest`, которая позволяет запускать только необходимую часть контекста Spring и выполнять запросы к эндпоинтам через объект `MockMvc`. Это обеспечивает изолированное и быстрое тестирование контроллеров, без необходимости запуска полной базы данных или сервера.

В случае необходимости интеграционного тестирования с подключением настоящей базы данных используется аннотация `@SpringBootTest` в сочетании с `@AutoConfigureMockMvc`. Такие тесты охватывают полный цикл обработки запроса — от получения HTTP-запроса до ответа от слоя данных.

Также были протестированы сценарии регистрации и входа в систему, включая генерацию и валидацию JWT-токенов, обработку ошибок при недопустимых данных, а также корректность авторизации при доступе к защищённым ресурсам.

Отчёт был сгенерирован с использованием плагина JaCoCo, что позволяет контролировать полноту покрытия кода тестами и выявлять неохваченные участки.

Таким образом, процесс тестирования охватывает все основные уровни архитектуры приложения: от отдельных методов и сервисов до REST-контроллеров. Это позволяет гарантировать корректность работы системы, устойчивость к ошибкам и надёжную реализацию бизнес-требований.

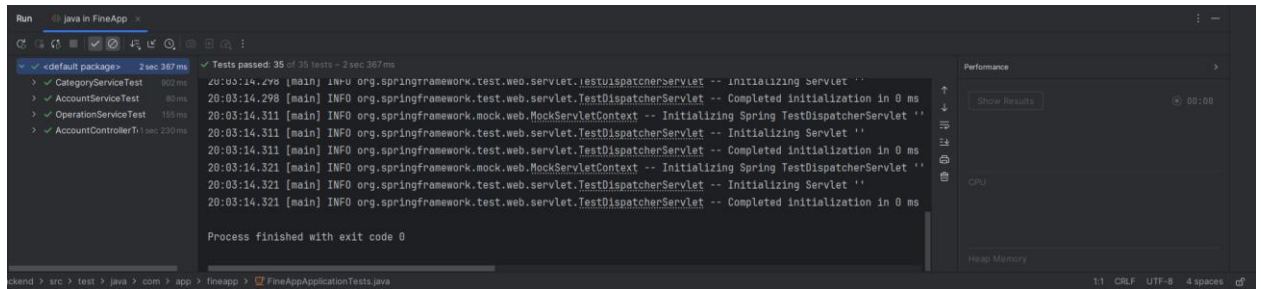


Рисунок 3.16 – Отчёт о тестировании приложения

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта было разработано и реализовано веб-приложение для учёта личных финансов, предназначенное для систематизации информации о счетах, операциях и категориях расходов и доходов. Приложение позволяет пользователю удобно управлять финансовыми данными, отслеживать денежные потоки и анализировать траты с учётом категорий.

В процессе разработки были успешно решены следующие задачи:

- Проведён анализ аналогичных решений, на основе которого определены основные функции, востребованные пользователями, и сформулированы требования к разрабатываемой системе;
- Разработана архитектура приложения с использованием принципов Onion Architecture, обеспечивающая модульность, расширяемость и изоляцию бизнес-логики от инфраструктурных компонентов;
- Реализован backend на базе Java и Spring Boot с использованием PostgreSQL в качестве основной базы данных, а также Redis для кеширования пользовательских данных и повышения производительности;
- Настроена аутентификация и авторизация с использованием JWT, что обеспечивает безопасность и удобство взаимодействия с системой;
- Спроектирован и реализован frontend на React с использованием Material UI. Интерфейс включает в себя страницы авторизации, управления аккаунтами, категориями и операциями, с поддержкой drag-and-drop и фильтрации данных;
- Обеспечена асинхронная обработка запросов, реализована система автологирования, а также возможность работы с приложением после восстановления сеанса;
- Проведено модульное и интеграционное тестирование ключевых компонентов системы с использованием JUnit и Mockito, что позволило выявить и устранить ошибки на ранних этапах разработки;

Разработанное веб-приложение ориентировано на широкую аудиторию пользователей и предоставляет удобный инструмент для повседневного ведения личной бухгалтерии. Гибкость архитектуры и применённые технологии позволяют в будущем легко расширять функциональность системы, интегрировать дополнительные сервисы, а также адаптировать приложение под мобильные платформы или корпоративные нужды.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Доманов, А. Т. Стандарт предприятия / А. Т. Доманов, Н. И. Сорока. – Минск: БГУИР, 2024. – 178 с.
- [2] Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения. – СПб.: Питер, 2018. — 352 с.
- [3] Документация по PostgreSQL [Электронный ресурс]. – Режим доступа: postgresql.org/docs/
- [4] Документация по Java Platform, Standard Edition [Электронный ресурс]. – Режим доступа: docs.oracle.com/en/java/javase/
- [5] Документация по Spring Framework [Электронный ресурс]. – Режим доступа: <https://spring.io/projects/spring-boot>
- [6] Паттерны объектно-ориентированного проектирования. / Гамма Э. [и др.]. – СПб.: Питер, 2020 – 448 с.
- [7] Документация по JWT (JSON Web Token) [Электронный ресурс]. – Режим доступа: jwt.io/introduction